A finite fair-graph certificate

and the SCC decision procedure for LTL, made proof-producing

Abstract

The standard automata-theoretic decision procedure for LTL satisfiability decomposes the state graph into strongly connected components: a non-trivial component is accepting when it meets every fairness set, and a fair path exists iff an accepting component is reachable. Its correctness rests on the assertion that an infinite path in a finite graph eventually remains within one component, which is infinitary and, in the negative direction, classical. We show that the assertion is not needed if the procedure is asked to return a certificate rather than a bit. The negative answer is taken to be a *well-founded record of a removal order* — at each step either a vertex with no out-edge, or a successor-closed block missing a fairness set — and this record both refutes the existence of a fair cycle, by two inductions on a finite closed trail, and serves as the induction principle from which Hilbert proofs are read off. The resulting statement is a theorem of graph theory with no formula in it; the temporal content enters only through the choice of fairness predicates. Everything has been checked in Agda, and the resulting procedure is compared, by checked equation, with the one of the companion note [1].

Introduction

The companion note [1] decides $\vdash_{\mathcal{L}} \varphi$ by an elimination procedure and reads a Hilbert proof off its failure. The present note does the same for the other standard presentation, the one that decomposes the graph into strongly connected components; the two are then in the same development and can be compared by computation rather than by assertion.

Three things came out of doing this that are not visible in either informal account.

First, the graph argument *factors*. Neither the elimination of [1] nor the usual SCC presentation separates the graph theory from the logic: sets of states, reachability, components and lassos are all described in terms of subformulas. They need not be. Section 1 states and proves a theorem about a finite graph, finitely many decidable vertex predicates and a vertex, in which no formula occurs; Sections 3 and 4 are the translation layer. The choice of predicate is where the logic goes, and it is one line (Definition 2.1).

Second, the "key insight" is a red herring in the direction it is usually stated. What is needed for correctness is not that an infinite path stays in a component, but the contrapositive statement about a *finite* cycle, and that is two inductions on a closed trail of the length it happens to have (Theorem 1.9). No infinite path is constructed and nothing is coinductive.

Third, the SCC form gives a genuine improvement on one point and a genuine loss on another, and both can be measured. The improvement: in a component that is bad for an eventuality, *every* state has that eventuality pending, so a single use of (Ind) refutes the whole component, where the elimination of [1] refutes one state per round. The loss: mutual reachability is quadratic where the elimination is one removing pass, and on the branch where the SCC decomposition is the whole story it is measurably slower (Section 7).

1 The graph layer

Throughout this section a *graph* is a finite list V of vertices together with a decidable edge relation G; a *fairness family* is a finite list $f_1, \dots, f_k$ of decidable predicates on vertices. No formula occurs anywhere in this section.

For a sublist $W \subseteq V$ write $\mathrm{out}_W(x) := \{y \in W : G(x, y)\}$.

Definition 1.1. A *trail* in W from u to w is a finite list $z_1, \dots, z_n$ of vertices of W with $G(u, z_1)$, $G(z_i, z_{i+1})$ and $z_n = w$; the empty trail runs from u to u. A *circuit at r in W for $f_1, \dots, f_k$* is a non-empty trail from r to r together with, for each i, a vertex on it satisfying f_i.

Indexing a trail by its endpoint rather than computing the endpoint with a last function is what keeps every concatenation below free of equational bookkeeping; the formalisation of [1] represents its paths the same way.

Definition 1.2 (Certificate)**.** The certificates $\mathrm{Cert}(W)$, indexed by the list W of vertices *still under consideration*, are generated inductively by

(C0) W is empty;

(C1) there is $x \in W$ with $\mathrm{out}_W(x) = \varnothing$, and a certificate for $W \setminus \{x\}$;

(C2) there are $C \subseteq W$ closed under G-successors lying in W, and an i with $f_i(z) = 0$ for every $z \in C$, and a certificate for $W \setminus C$.

A certificate is a finite object: it is the removal order, with a reason attached to each step. It is *not* a decomposition into components — no component object, no quotient and no equality of components occurs — and (C2) does not require C to be strongly connected, only successor-closed.

Lemma 1.3 (Induction principle)**.** *Let R be a property of vertices. Suppose*

(H1) for every W and every $x \in W$ with $\mathrm{out}_W(x) = \varnothing$: if R holds of every vertex outside W, then $R(x)$;

(H2) for every W, every $C \subseteq W$ closed under G-successors in W and every i with f_i false throughout C: if R holds of every vertex outside W, then R holds of every $z \in C$.

Then a certificate for W, together with R outside W, gives R of every vertex.

Proof. Induction on the certificate. In case (C0) nothing is left, so the hypothesis already gives R everywhere. In case (C1) apply (H1) and note that a vertex outside $W \setminus \{x\}$ is outside W or is x. Case (C2) is the same with (H2) and C in place of x. □

Lemma 1.3 is the whole reason the certificate is defined as it is: (H1) and (H2) are, on the nose, cases (R1) and (R2) of [1, Def. 3.1], with the formulas removed. That is made precise in Section 4.

Computing one

Definition 1.4. $\mathrm{Reach}_W(v)$ is computed by a *growing* search: start from the empty accumulator; a round adds every vertex of W that is wanted — is v, or is a G-successor of something already accumulated — and is not yet there; stop when a round adds nothing.

The stopping condition, not a bound, is what gives completeness: when a round adds nothing the accumulator contains v and is closed under successors in W, and an induction along a trail then shows it contains everything reachable. There is no fuel and no claim that $|W|$ rounds suffice. Termination is by the inductive finiteness of vertices: one cannot go on choosing distinct vertices from a fixed finite list. This is how the formalisation of [1] terminates all of its searches, applied here to the one computation that note deliberately avoided: [1, Rem. 3.2] reads (R2) as a greatest fixed point precisely so as not to compute reachability.

Definition 1.5. $\mathrm{scc}_W(r) := \{w \in \mathrm{Reach}_W(r) : r \in \mathrm{Reach}_W(w)\}$, and r is *terminal* in W when $\mathrm{Reach}_W(r) = \mathrm{scc}_W(r)$, i.e. when everything r reaches reaches r back.

Both are decidable, and the condensation is never built: a condensation edge is followed by *moving the representative*, and terminality is one boolean.

Lemma 1.6. *A non-empty W contains a terminal representative.*

Proof. While r is not terminal there is $w \in \mathrm{Reach}_W(r)$ with $r \notin \mathrm{Reach}_W(w)$; move the representative to w. The search terminates because $\mathrm{Reach}_W(w)$ avoids r, so each move gives up one vertex of the ambient list, and one cannot go on giving up distinct vertices. The invariant carried is a single hypothesis — every vertex of the current reachable set still satisfies the bar predicate — and no list of visited representatives is needed; that acyclicity of the condensation makes the components visited distinct is thereby not a lemma but the termination of this search. □

Lemma 1.7 (One step)**.** *Suppose every $x \in W$ has an out-edge in W. Then either W is empty, or its terminal component is fair and a circuit is built in it, or its terminal component misses some f_i everywhere.*

Proof. Take r terminal by Lemma 1.6 and $C := \mathrm{scc}_W(r)$. If some f_i is false throughout C we are in the third case. Otherwise each f_i has a witness $w_i \in C$. Since r has an out-edge $x \in W$ and r reaches x, terminality sends a trail from x back to r, so there is a non-empty closed trail c_0 at r. Now add one *leg* per fairness predicate: the leg for f_i is c_0 followed by a trail $r \to w_i$ inside C followed by a trail $w_i \to r$ inside C, both of which exist because C is a component. Carrying c_0 in front makes the first half non-empty, so w_i is a vertex the leg *visits* and not merely an endpoint — which matters when $w_i = r$. The concatenation of the legs with c_0 is a circuit at r. □

Theorem 1.8 (Fair-graph certificate)**.** *Given a graph, a fairness family and a vertex $v \in V$, one can compute either*

(a) a lasso from v — a trail from v to some r, and a circuit at r — whose cycle meets every f_i, or

(b) a certificate for $\mathrm{Reach}_V(v)$.

Proof. Run the following removing iteration on $W := \mathrm{Reach}_V(v)$. Ask whether some $x \in W$ has $\mathrm{out}_W(x) = \varnothing$. If so, recurse on $W \setminus \{x\}$; a lasso found there is a lasso in W, and a certificate found there is extended by (C1). If not, apply Lemma 1.7: the empty case gives (C0), the fair case gives a circuit, and the unfair case gives a C and an i, and we recurse on $W \setminus C$, extending by (C2). Each step removes at least one vertex — for (C2) the representative r, which lies in its own component — so the iteration terminates. Finally a circuit at r inside $\mathrm{Reach}_V(v)$ is joined to v by the trail that put r in the reachable set. □

Note that no rank function and no topological sort appear: the iteration removes, so termination is by shrinking, and the only thing that needed proof is Lemma 1.6.

The two answers are exclusive

This is the half that replaces the lecture's "key insight", and the point is that it comes out finitary.

Theorem 1.9. *A certificate for W forbids a circuit inside W.*

Proof. Induction on the certificate; each step shows the circuit *survives* the removal, so it is still there when nothing is left, which (C0) forbids.

For (C1): a vertex on a cycle has a successor — along a trail, every vertex visited has a successor provided the endpoint has, and for a closed trail the endpoint is the start, whose successor is the trail's own first step — so a sink lies on no cycle, and the cycle avoids x.

For (C2): any vertex of a cycle reaches any other — take the suffix of the trail to the base point and then the prefix out again — and a successor-closed block that a trail starts in contains everything the trail reaches. So if the cycle met C it would lie inside C, and then its witness for f_i would satisfy an f_i that is false throughout C. Hence the cycle avoids C. □

Corollary 1.10. *The negative answer of Theorem 1.8 is a decision: if a fair lasso from v exists at all, answer (a) is returned.*

Proof. A lasso from v has all of its cycle inside $\mathrm{Reach}_V(v)$, by the completeness of Definition 1.4; so a certificate for $\mathrm{Reach}_V(v)$ would contradict Theorem 1.9. □

Both branches are inhabited and neither theorem is vacuous: the graph $p \to q$, $q \to s$, $s \to q$ with the two predicates "is q", "is s" exhibits a circuit, and with the single predicate "is p" exhibits a certificate, whose removal order is (C2), (C1), (C0) — the recurrent component $\{q, s\}$ is refuted first, and only then is p a sink. That is the backwards induction in miniature: p is refuted *because* its successors already were.

2 Where relevance goes

Fix φ and let $\Box A_1, \ldots, \Box A_m$ enumerate the members of $\mathrm{Sub}(\varphi)$ of the form $\Box A$, as in [1]. The graph is taken to be the states with the transition relation of [1, Def. 2.3].

The informal argument quantifies over "the fairness sets *relevant* in C", which is well defined only because a pending eventuality is uniform on a component: if $\Box A \notin v$ for one $v \in C$ then, by persistence along a path plus strong connectivity, $\Box A \notin w$ for every $w \in C$. We do not need that lemma, because relevance can be put inside the predicate.

Definition 2.1. For each j let

$$f_j(z) \;:=\; [\,\Box A_j \in z\,] \;\vee\; [\,A_j \notin z\,],$$

that is: the j-th eventuality is not pending at z, or it is fulfilled at z.

A component in which the eventuality is not pending meets f_j for free, and a component in which it is pending meets f_j exactly by fulfilling it. So the graph layer of Section 1 knows only "C meets every f", and uniformity on a component is never needed. The two readings of the same definition are what Sections 3 and 4 use: its *truth* at a witness is fulfilment, and its *falsity* throughout a block is exactly the pair of hypotheses of case (R2).

3 The lasso branch

A trail in the state graph *is* a path in the sense of [1, §5]; the only work is the cycle-fulfilment property that Proposition 3.2 below requires.

Lemma 3.1 (Persistence to a visited state)**.** *Let $\Box A \notin v$ and let z be a state visited by a path $v \to \cdots \to x$. Then either the eventuality is fulfilled at some state visited, or $\Box A \notin z$.*

Proof. The induction of [1, Lem. 2.8] propagated along the path, with the target carried along and the path cut short when it is reached. □

The corresponding statement for the *endpoint* is the one [1] needs; the witness that Theorem 1.8 supplies lies somewhere on the cycle and not at its base point, which is why the visited-state form is needed here.

Proposition 3.2. *A fair lasso in the sense of Theorem 1.8, for the predicates of Definition 2.1 and inside any set of states contained in U_∞, is a fulfilling lasso in the sense of [1, Prop. 5.3].*

Proof. The stem and the cycle are paths, and the cycle is non-empty, so only the cycle-fulfilment property has to be produced: for a state s on the cycle at which $\Box A_j$ is pending, a state of the cycle at which A_j fails. Travel along the cycle from s to the base point r: by [1, Lem. 2.8] either A_j fails on the way — and we are done — or $\Box A_j \notin r$. In the latter case go out from r to the witness w that Theorem 1.8 chose for f_j: by Lemma 3.1, either A_j fails on the way, or $\Box A_j \notin w$, and then the first disjunct of $f_j(w)$ is false, so the second holds and $A_j \notin w$. □

So the positive branch feeds the unchanged tail of [1]: the lasso is unwound to a run, Hintikka's lemma evaluates $\neg\varphi$ to 1 at the start, and soundness turns any derivation into a contradiction.

Remark 3.3. The converse also holds, and is worth stating because it makes Proposition 3.2 unconditional. A fulfilling lasso is a fair lasso for the f_j: if the eventuality is not pending at the base point, the base point is its own witness — and it lies *on* the cycle, the cycle being non-empty — and if it is pending, the lasso's own cycle-fulfilment property supplies a state of the cycle at which A_j fails, which is the other disjunct. Hence, by [1, Prop. 5.3] and Corollary 1.10, Theorem 1.8 run on U_∞ *always* returns answer (a), and one obtains a second proof of [1, Prop. 5.3] in which no round and no pigeonhole occurs.

4 The refutation branch

Take $R(v) := \vdash_{\mathcal{L}} \neg\chi_v$ in Lemma 1.3. Then the hypothesis carried — R holds outside W — is the induction hypothesis of [1, Thm. 4.1] verbatim, and the two hypotheses to be discharged are its two cases.

(H1) is case (R1). The out-neighbours of x inside W are the $\to$-successors of x inside W, so "$\mathrm{out}_W(x) = \varnothing$" *is* the hypothesis of (R1). The proof is [1, Thm. 4.1, case (R1)] unchanged: the step lemma offers all state-successors, each is refuted by the induction hypothesis, (D1) puts the refutation under $\bigcirc$ and (Lin) brings the negation out.

(H2) is case (R2), improved. Let C be the block and f_j the predicate it misses. Then for every $z \in C$

$$f_j(z) = 0 \qquad \text{i.e.} \qquad \Box A_j \notin z \quad \text{and} \quad A_j \in z,$$

which are the two hypotheses (R2) needs, *at every state of* C rather than at one state per round. Take $B := \beta_C$; then

(i) $\vdash_{\mathcal{L}} B \to A_j$, since $A_j \in z$ throughout C;

(ii) $\vdash_{\mathcal{L}} B \to \bigcirc B$, by the step lemma at each $z \in C$, dropping the successors outside W by the induction hypothesis and keeping the rest because C is closed under successors lying in W;

(iii) $\vdash_{\mathcal{L}} B \to \Box A_j$, by (Ind) through (D5) and (D3).

Since every $z \in C$ has $\Box A_j$ pending, $\vdash_{\mathcal{L}} \chi_z \to \neg \Box A_j$, and $\vdash_{\mathcal{L}} \chi_z \to B$; so $\vdash_{\mathcal{L}} \neg \chi_z$ for *every* $z \in C$ from a single use of (Ind).

This is the improvement the SCC form buys. In [1] the greatest fixed point W_j is recomputed each round and refutes the states pending in it one round at a time; here the whole component leaves at once, with one invariant.

Remark 4.1. The reader will notice that Lemma 1.3 quantifies (H1) and (H2) over an arbitrary W and knows nothing about states — it is the graph layer's induction principle and has no business knowing. The formalisation therefore intersects the block with the ambient state set on the spot; the intersection is still successor-closed, because the state set is. It is one filter and no new theory.

5 Two lemmas at the seam

The two branches want different sets. The refutation branch is stated against the set the search explores — in the formalisation $\mathrm{U}(\varphi)$, the $\to$-closure of the states containing φ, which replaces $\mathrm{St}(\varphi)$ for the search but not for the derivations — while the lasso branch delivers into machinery stated against U_∞. Two lemmas bridge them, and neither is in [1].

Lemma 5.1. *Every state of* $\mathrm{U}(\varphi)$ *is reached by a path from a state containing* φ.

Proof. The closure is computed by the growing search of Definition 1.4 with the seed changed; the same recursion shows that a state a round adds is either a seed — and then the empty path does it — or a successor of something already accumulated, and then that thing's path grows by one step. □

Lemma 5.1 says the closure contains *nothing else*, which is what makes "a fair component anywhere in $\mathrm{U}(\varphi)$" the right question to ask: without it the positive branch would exhibit a loop that is a countermodel to nothing.

Definition 5.2. A set C of states is a *trap* if every $z \in C$ has a $\to$-successor in C, and for every $z \in C$ and every j pending at z there is a path inside C from z to a state fulfilling the j-th eventuality.

Lemma 5.3. *A trap* $C \subseteq \mathrm{St}(\varphi)$ *contained in* $\mathrm{U}(\varphi)$ *is contained in* U_∞.

Proof. The two clauses of a trap are the negations of (R1) and (R2) read as properties of a *set*: if $C \subseteq U_k$ then no $z \in C$ is removed by (R1), having a successor in $C \subseteq U_k$; and none is removed by (R2), since a path inside $C \subseteq U_k$ to a fulfilling state contradicts the soundness of the greatest fixed point of [1, Rem. 3.2]. So $C \subseteq U_{k+1}$, and the claim follows by induction along the elimination. □

Lemma 5.3 is one direction of the completeness of the elimination: it says the elimination throws nothing useful away. The other direction — that U_∞ is itself a trap — is [1, Lem. 5.1]. Note that the vertex set of a fair lasso is a trap: every vertex has the next one as a successor, the base point has the cycle's first step, and a pending eventuality is fulfilled along the walk to the witness by Proposition 3.2's argument.

6 The decision procedure

Theorem 6.1. *There is a total computable function taking φ to an element of*

$$\mathrm{Deriv}_{\mathcal{L}}(\varphi) \;+\; \mathrm{LassoCountermodel}(\varphi),$$

which decides by asking which answer Theorem 1.8 gives on $\mathrm{U}(\neg\varphi)$.

Proof. Run the iteration of Theorem 1.8 on the whole of $\mathrm{U}(\neg\varphi)$ — not from a single vertex, since the refutation branch must account for every state at once — with the predicates of Definition 2.1.

If it returns a certificate for $\mathrm{U}(\neg\varphi)$, apply Lemma 1.3 with $R(v) := \vdash_{\mathcal{L}} \neg\chi_v$ and Section 4; the hypothesis outside $\mathrm{U}(\neg\varphi)$ is vacuous. Every state of $\mathrm{U}(\neg\varphi)$ is then refutable, and a state outside it cannot contain $\neg\varphi$, so $\vdash_{\mathcal{L}} \chi_v \to \varphi$ for all of them by (L2); conclude by [1, Lem. 2.6] and (L3),(L5). Observe that the elimination of [1] is not run at all on this branch.

If it returns a fair component at r, take by Lemma 5.1 a state s containing $\neg\varphi$ and a path $s \to \cdots \to r$; stem and cycle together are a fair lasso from s. Its vertex set is a trap, so by Lemma 5.3 it lies in U_∞, and Proposition 3.2 turns it into a fulfilling lasso through s. Hintikka's lemma and soundness then give the countermodel exactly as in [1, Thm. 6.1]. □

The two branches are asymmetric in the same way as in [1]: the certificate branch returns a derivation and never mentions a lasso, and the fair branch returns a lasso and mentions a derivation only to consume one. Note the asymmetry in cost as well. On the certificate branch the elimination is never computed; on the fair branch it *is* computed, by Lemma 5.3, but it is not what decides and it produces no lasso.

7 Comparison

The two procedures are in one development, so they can be compared by checked equation rather than by running them and looking.

Agreement. On the forty-six statements of Figure 11 of [3], together with $\Box\Diamond(p \vee q) \to (\Box\Diamond p \vee \Box\Diamond q)$, the two procedures take the same branch; each of the forty-seven equations $\mathrm{isProof}(\mathrm{decideSCC}\, X) = \mathrm{isProof}(\mathrm{decide}\, X)$ is checked. Twenty-two are proofs and twenty-five are countermodels.

Different lassos. For $\Box(p \vee q) \to (\Box p \vee \Box q)$ the two countermodels differ:

	start	stem	cycle
decide	$p \wedge q$, neither forever	3	1
decideSCC	q, not p	2	4

Both are countermodels and each carries its own refutation of derivability. The cycle of length four is not an accident: the construction of Lemma 1.7 prepends one leg per fairness predicate to the bare loop, and Sub of the negation has three eventualities.

Cost. Measured cold, with the import baseline subtracted:

	decide	decideSCC
$\Box(p \vee q) \to (\Box p \vee \Box q)$	0.19 s	0.18 s
$\Box\Diamond(p \vee q) \to (\Box\Diamond p \vee \Box\Diamond q)$	0.78 s	1.24 s

The first is decided on the countermodel branch and the second on the proof branch. The prediction that the SCC form is slower — mutual reachability being quadratic in the component where the greatest fixed point is one removing pass — is confirmed on the proof branch, where it is 1.6 times slower, and refuted on the countermodel branch, where the two are level: there the SCC procedure pays for both the graph search and Lemma 5.3, but saves the round construction and the boundary search of [1, Prop. 5.3], and on that formula the two cancel.

8 Remarks

Remark 8.1 (What the phase buys, and what it does not)**.** It does not buy constructivity: [1] is already constructive, and its greatest fixed point is arguably the SCC decomposition performed without decomposing — the union of the bad recurrent regions, computed by a removing iteration in one pass. It does not buy speed. What it buys is the factoring of Section 1, an explanation of *why* (Ind) appears at all, and the refutation of a whole component by one invariant.

Remark 8.2 (Absences)**.** No infinite path is constructed; nothing is coinductive; there is no pigeonhole lemma; there is no rank function and no topological sort; there is no component object, no quotient and no equality of components; and the condensation is never built. Definition 1.4 is the one computation that [1, Rem. 3.2] sidesteps, and it is done without cardinality theory.

Remark 8.3 (A worked example, and a correction)**.** The appendix of the lecture notes [2] analyses $\Diamond a \wedge \Diamond b \to \Diamond(a \wedge b)$ by hand: it lists the locally consistent states with $\Diamond(a \wedge b) = 0$, their out-edges, and a countermodel path. The graph agrees with the one computed here exactly — eight states with the same valuations, twenty-four edges, the same out-degrees — although the two are obtained differently: the hand analysis *assumes* $\Diamond(a \wedge b) = 0$ to cut its table down, whereas here the eight states are the closure of the states containing the negation and $\Diamond(a \wedge b) = 0$ falls out.

The countermodel path given there, $s_3 \to s_5 \to s_7 \to s_7 \to \cdots$ looping at s_7, is however *not fair*: at s_7 one has $\Diamond b = 1$ and $b = 0$, so s_7 lies outside the very fairness set $F_{\Diamond b}$ that the notes define, and the loop promises b forever without delivering it. In the notation of Definition 2.1 this is the checked equation $f_{\Diamond b}(s_7) = 0$. The path presumably intended is $s_3 \to s_5 \to s_7 \to s_5 \to \cdots$, alternating so that b recurs at s_5 and a at s_7; both edges exist. The elimination procedure of [1] returns instead $s_7 \to s_4 \to s_0 \to s_0 \to \cdots$: a at stage 0, b at stage 1, and everything false from stage 2 on. It ends in a state where nothing is pending at all, which the hand analysis never considers because it is looking for a loop that *keeps* the eventualities alive. The procedure of Theorem 6.1 also answers this formula with a countermodel; the formalisation checks that it does, but not which lasso it returns.

Remark 8.4 (Formalisation)**.** Everything above has been checked in Agda — `--safe`, `--without-K`, `--exact-split`, no library, no postulates, no unfilled goals, no termination pragmas, and no `with` — beside the development of [1] and sharing its step lemma, its case (R1), its notion of run and its soundness. The new modules are

`LTL.Graph`	Section 1	1077	0.60 s
`LTL.SccLasso`	Sections 2, 3	372	0.34 s
`LTL.SccExtract`	Section 4	380	0.32 s
`LTL.SccTrap`	Section 5	289	0.21 s
`LTL.DecideSCC`	Section 6	256	0.30 s
`LTL.Compare`	Section 7	408	20.7 s
`LTL.Lecture`	Remark 8.3	184	10.2 s

lines and cold check time; the last two are benchmarks, being forty-seven decision problems and one worked example respectively. `LTL.Graph` is parameterised by the type of vertices and a sound, reflexive equality test on it, and imports nothing about formulas or valuations; `LTL.SccLasso` instantiates it at valuations, and `LTL.GraphExamples` runs its checks on natural numbers.

Three places where formalising changed the argument rather than transcribing it. Lemma 3.1, which the informal argument does not distinguish from persistence to the endpoint, and which is needed because the witness sits on the cycle rather than at its base point. Lemma 5.1, which the closure construction of [1] did not have — it proved that the closure contains the seeds and is closed, but not that it contains nothing else — and without which the fair branch exhibits a loop that is a countermodel to nothing. And Lemma 5.3, which the informal accounts do not mention at all because they do not have two state sets to bridge; it is the one place where the SCC procedure must still compute the elimination of [1].

Remark 8.5. The construction is stated for the $\Box$-only syntax of [1], where persistence is (T3) and is already proved. Extending it to *until* faces exactly the obligation recorded at the end of [1], and one more: the fairness predicate of Definition 2.1 would become $[\,A \,\mathcal{U}\, B \notin z\,] \vee [\,B \in z\,]$, and the argument of Proposition 3.2 would have to carry the side condition that A holds along the way.

References

[1] *Proof extraction from a failed LTL tableau: a uniform loop-invariant construction*, companion note.

[2] *A Complete Decision Algorithm for Linear Temporal Logic*, lecture notes.

[3] P. H. Schmitt and J. Goubault-Larrecq, *Linear Temporal Logic*, Figure 11.